

Base de Datos Clave - Valor

Clave - Key

70248160-N

32123844-Z

...

233456879-A

Valor - Value

Juan Antonio Pérez natural de Villa del Mar, estudio en las mejores universidades del EEUU y ha sido padre recientemente.

Manuel Rodríguez es fontanero de Olmedo y no esta casado pero le encantaría encontrar pareja.

...

Paloma García es una importante escritora de literatura infantil muy querida en su ciudad natal.

Características Generales

- Es un tipo de base de datos no relacional que utiliza un método simple de **clave-valor** para almacenar datos.
- Tanto las claves como los valores pueden ser cualquier cosa, desde valores simples hasta estructuras compuestas complejas.
- Son altamente divisibles y permiten el escalado horizontal a escalas que otros tipos de bases de datos no pueden alcanzar.
- Similar a una tabla hash simple.
- Indicada cuando todos los accesos a la base de datos se hacen por una clave
- El valor no es analizado por el producto, lo trata como una unidad

La Clave

- Identificador único que le permite acceder al valor asociado con esa clave y debe ser único.
- En teoría la clave podría ser cualquier cosa, pero depende de cada KVDBMS ya que uno puede imponer limitaciones mientras que otro ninguna.
- Productos como Redis tienen un tamaño de clave máximo permitido es 512 MB.
- Puede usar cualquier secuencia binaria como clave, desde una secuencia de texto corta hasta el contenido de un archivo de imagen. Incluso la cadena vacía es una clave válida.
- claves muy largas consumen memoria en la RAM y aumentan el costo de la comparación de cadenas, se busca un balance entre descriptivo y conciso.

Diseñando las Claves

- La selección se complica si queremos guardar información de varias entidades con mismo nombre de clave (por ejemplo: número de cliente, número de proveedor, etc.)
- Una estrategia es añadir a las claves un prefijo con el nombre de la entidad. La convención estándar en Redis para estructurar claves es utilizar dos puntos, por ejemplo **cliente:Numero** o **proveedor:Numero**.
- Otro punto importante es adoptar una convención única de escritura (usualmente snake_case) en todo el sistema para evitar colisiones accidentales (user:1000 vs User:1000).

El Valor

- El valor puede ser cualquier cosa binaria, productos como Redis aceptan hasta 512MB (texto, números, HTML, código de programación, una imagen, etc.)
- El valor también podría ser una lista, o incluso otro par clave-valor encapsulado en un objeto.
- Algunos KVDBMS le permiten especificar un tipo de datos para el valor.

Diseño

- **Tener en cuenta:**
- **Cómo estructurar las claves:** Un buen diseño facilita la definición de los conceptos.
- **Qué información capturar en los valores:** Solo aquellos que son necesarios para el objetivo.
- **Introducir abstracciones:** Patrones de diseño para crear estructuras de más alto nivel que los simples pares Clave-Valor

Abstracción

cualquier técnica que permita trabajar con los datos de una manera más abstracta y simplificada.

- Significa usar estructuras de datos avanzadas (como Hashes, Sets o Sorted Sets) para simular comportamientos que normalmente tendríamos en bases de datos relacionales.
- El patrón de "**Objeto**" o "**Entidad**" (Usando Hashes): En lugar de crear múltiples claves simples para representar las propiedades de una entidad, las agrupamos en una sola estructura.

- Sin abstracción:

usuario:1000:nombre = "Carlos";

usuario:1000:email = carlos@mail.com

- Con abstracción (Hash):

usuario:1000 = { nombre: "Carlos",
email: "carlos@mail.com",
plan: "premium" }

menos memoria y puedes obtener todo el perfil del usuario con una sola consulta.

Abstracción

2. El patrón de "**Índice Secundario**" (apuntador) Solo podemos buscar si conocemos la clave exacta, ¿qué pasa si necesitas buscar al usuario 1000 por su correo electrónico?

Creamos una clave especial que funciona como un apuntador, primero buscamos el apuntador y después el valor.

usuario:email:carlos@mail.com con Valor: 1000

buscamos la clave **usuario:email:carlos@mail.com** y luego buscamos el perfil completo con la clave **usuario:1000**.

3. El patrón de "**Relaciones**" (Sets) Tengo un usuario que sigue a otros usuarios. Usando Sets, la clave

usuario:1000:seguidores: [24, 85, 912]

Podemos determinar la presencia de un elemento en un set o la intersección, diferencia etc.

Abstracción

4. El patrón de "**Tabla de Clasificación**" (SortedSets):
Si quiero los productos más vendidos, un ordenamiento sería muy lento.

Un SortedSet permite a cada elemento un valor asociado.

productos:ventas_globales:("Pantalones Jean"=5400 ,
"Camisas Lisas"=7200,....)

se ordenan automáticamente al insertarlos y se pueden obtener slices de este conjunto.

Ventajas

- Las bases de datos clave-valor son muy efectivas en la consulta y fáciles de escalar.
- Al no exigir ningún esquema fijo, se pueden realizar modificaciones en la base de datos mientras se realizan acciones en otras entradas.
- La gran velocidad de búsqueda de las estructuras clave / valor y la capacidad de brindar alta velocidad con un gran volumen de datos.
- La información está dispuesta de forma clara.

Desventajas

- Solo se puede consultar un valor a través de una clave específica, no contempla otro método de acceso.
- Las búsquedas complejas sobre un único concepto no están disponibles para recuperar por contenido.
- Las búsquedas complejas generalmente requieren modelado adicional, índices secundarios o procesamiento en la aplicación o mediante algún lenguaje de scripting (LUA).

Cuando Usarlas



Almacenar información de sesión

- El sessionId de la sesión web de un usuario.
- Todo en una sesión puede almacenarse con un solo PUT o ser recuperado con un solo GET
- Toda la información de la sesión se almacena en un único objeto

Cuando Usarlas



Perfiles de Usuarios/Preferencias

- Cada usuario tiene un único user Id username
- Todos los atributos asociados al usuario (paleta de colores, zona horaria, etc.) pueden recuperarse en un único acceso
- Todos esos atributos pueden almacenarse en un único objeto

Cuando Usarlas



Carros de Compra

- Los sitios de **e-commerce** tienen carros de compra asociados al usuario.
- Toda la información del usuario asociado al carro de compra puede almacenarse como un único objeto.



Cuando NO Usarlas

- Consultas que relacionen datos.
- Transacciones complejas con consistencia ACID estricta.
- Consulta por los valores almacenados dentro del campo valor
- Operaciones que abarcan diferentes conjuntos de datos

Ranking de las Bases Clave-Valor

Rank			DBMS	Database Model	Score		
Aug 2026	Jul 2026	Aug 2025			Aug 2026	Jul 2026	Aug 2025
1.	1.	1.	Redis	Key-value, Multi-model 	156.51	+2.70	+9.32
2.	2.	2.	Amazon DynamoDB	Multi-model 	54.19	-1.73	-29.29
3.	3.	3.	Microsoft Azure Cosmos DB	Multi-model 	20.67	-0.56	-2.17
4.	4.	4.	Memcached	Key-value	14.26	+0.22	-1.90
5.	5.	5.	etcd	Key-value	7.10	-0.18	+0.22
6.	6.	↑ 7.	Hazelcast	Key-value, Multi-model 	4.61	-0.10	-0.09
7.	7.	↓ 6.	Aerospike	Multi-model 	4.33	+0.36	-0.50
8.	8.	↑ 12.	RocksDB	Key-value	3.94	+0.10	+1.02
9.	↑ 10.	↓ 8.	Oracle NoSQL	Multi-model 	3.49	-0.05	+0.04
10.	↓ 9.	↑ 13.	Arango 	Multi-model 	3.46	-0.08	+0.55



- Redis es un almacén de datos en memoria
- Se utiliza como base de datos, caché, streaming, pub/sub, IoT y microservicios.
- Posee estructuras de datos como cadenas, hashes, listas, conjuntos, conjuntos ordenados con consultas de rango, mapas de bits, etc.
- Puede ejecutar operaciones atómicas sobre estos tipos.
- Puede calcular operaciones sobre conjuntos o conseguir el miembro con la clasificación más alta en un conjunto ordenado, etc.



Posee:

- Replicación integrada
- Admite secuencias de comandos LUA
- Liberación de claves temporales
- Transacciones
- diferentes niveles de persistencia
- Alta disponibilidad (***Redis Sentinel***)
- Partición automática (***Redis Cluster***).



- Trabaja completamente con un conjunto de datos en memoria.
- Puede convertir sus datos en permanentes mediante el volcado periódico de datos en el disco (datos o registro de operaciones).
- También puede deshabilitar la persistencia si solo necesita una memoria caché.
- Redis admite la replicación asíncrona, con sincronización rápida sin bloqueo y reconexión automática con resincronización parcial en particiones de red.

Quienes usan Redis



GitHub



Snapchat

Hulu

by Hulu



StackOverflow



Craigslis

<https://techstacks.io/tech/redis>

TikTok

by ByteDance Ltd.

[Otros sitios que usan Redis...](#)

Quienes usan Redis

- **Empresas de tecnología:** Twitter/X, GitHub, Pinterest y Snapchat lo emplean para almacenamiento en caché de alto rendimiento, gestión de sesiones distribuidas y contadores en tiempo real
- **Empresas de medios sociales:** Instagram y TikTok lo utilizan para acelerar la carga de contenidos (*caching* de datos), persistencia temporal de sesiones de usuario y analítica de interacciones en vivo.
- **Empresas de comercio electrónico:** Amazon, eBay y Shopify lo aprovechan en el manejo de carritos de compra, gestión de catálogos en memoria y motores de recomendación de baja latencia.
- **Empresas de juegos:** ynga y Electronic Arts lo aplican para tablas de clasificación (*leaderboards*), almacenamiento del estado del juego en tiempo real y sincronización en partidas multijugador.
- **Finanzas y servicios bancarios:** Entidades financieras y pasarelas de pago lo adoptan para la detección de fraudes en tiempo real, validación instantánea de transacciones y control de tasa de peticiones (*rate limiting*).

Replicación en Redis

Proceso que permite crear copias exactas de un servidor llamado "**maestro**" en uno o varios servidores llamados "**esclavos**", proporcionando redundancia y escalabilidad al sistema. El mecanismo es:

- Empezamos estableciendo el servidor maestro.
- Luego configuramos los servidores esclavos para conectarse al servidor maestro.
- El servidor maestro envía los datos que cambian a los servidores esclavos a través de un flujo de datos TCP utilizando el protocolo de replicación de Redis.
- Cuando un esclavo se conecta a un maestro por primera vez (**inicialización**) el maestro envía una copia completa de los datos (**snapshot**) y los comandos que se ejecutaron en el maestro desde el inicio del proceso hasta el momento en que se finalizó la copia.

Replicación en Redis: Escenarios

- Cuando las instancias maestra y esclava están bien conectadas, el maestro mantiene el esclavo actualizado enviando una secuencia de comandos al esclavo para replicar los efectos de las acciones que cambiaron el conjunto de datos.
- Cuando el enlace entre el maestro y el esclavo se rompe, el esclavo intentará volver a conectarse y proceder con una resincronización parcial.
- Cuando no es posible, el esclavo solicitará una resincronización completa, esto significa que el maestro creará una instantánea la enviará al esclavo y continuará enviando la secuencia de comandos a medida que el conjunto de datos cambia.

Persistencia en Redis

Redis proporciona distintas formas de persistencia:

- La persistencia de tipo RDB que realiza snapshots de su conjunto de datos en intervalos especificados.
- La persistencia de AOF registra cada operación de escritura recibida en el master, permitiendo reconstruir el conjunto de datos.

[Persistencia en Redis](#)

Persistencia en Redis

- Redis puede sobrescribir en segundo plano el registro cuando se vuelve demasiado grande.
- Se puede desactivar la persistencia en absoluto si lo que desea es que los datos existan solo en el servidor mientras se está ejecutando.
- Es posible combinar AOF y RDB en la misma instancia.
- Cuando se reinicia Redis, si el archivo existe AOF se utilizará para reconstruir el conjunto de datos ya que se garantiza que será el más completo.

Persistencia en Redis – Ventajas RDB

- RDB es una representación muy compacta de un solo punto en un solo archivo de sus datos de Redis.
- Los archivos RDB son perfectos para copias de seguridad. Por ejemplo, archivar copias RDB cada hora durante 24 horas y guardar una instantánea RDB todos los días durante 30 días.
- RDB es muy bueno para la recuperación de desastres, ya que un único archivo compacto se puede transferir a centros de datos lejanos.
- RDB maximiza el rendimiento de Redis ya que el proceso principal no hace I/O.
- RDB permite reinicios más rápidos con grandes conjuntos de datos en comparación con AOF.

Persistencia en Redis – Desventajas RDB

- RDB NO es bueno para minimizar la posibilidad de pérdida de datos en caso de que Redis deje de funcionar (corte de suministro energía).
- Se debe estar preparado para perder los últimos minutos de datos.
- Si bien RDB utilizan un proceso secundario, este puede consumir mucho tiempo si el conjunto de datos es grande generando demoras en el maestro.

Persistencia en Redis – Configuración AOF

- AOF Redis puede tener diferentes políticas de sincronización (fsync):
 - ***always***: Escribe cada comando en el archivo AOF tan pronto como se ejecuta. (impacto en el rendimiento)
 - ***everysec***: Escribe los comandos en el archivo AOF de forma asincrónica, pero al menos una vez por segundo (predeterminada)
 - ***no***: Escribe de forma asincrónica sin ningún tipo de sincronización con el disco. (mejor rendimiento, pero menor nivel de durabilidad)
- "everysec", utiliza un hilo en background y el hilo principal intentará realizar escrituras cuando no hay ninguna operación en progreso, lo máximo que se perdería sería un segundo.

Persistencia en Redis – Ventajas AOF

- El registro de AOF es un registro de append. Incluso si el registro finaliza con un comando medio escrito por algún motivo puede repararlo fácilmente.
- Cuando se vuelve demasiado grande mientras continúa añadiendo al archivo anterior, produce uno nuevo con el conjunto mínimo de operaciones necesarias para crear el conjunto de datos actual y cuando este segundo archivo está listo los cambia y comienza en el nuevo.
- AOF contiene un registro de todas las operaciones de manera secuencial en un formato fácil de entender y analizar. Incluso puede exportar fácilmente.

Persistencia en Redis – Desventajas AOF

- Los archivos AOF suelen ser más grandes que los archivos RDB para el mismo conjunto de datos.
- AOF puede ser más lento que RDB. Con "**fsync** = **no**" es tan rápido como RDB.
- AOF trabaja actualizando incrementalmente un estado existente, como MySQL o MongoDB, RDB crea todo desde cero una y otra vez, eso es conceptualmente más robusto.

Comparativa

Comparación entre RDB y AOF en Redis

Característica	RDB (Redis Database)	AOF (Append Only File)
Mecanismo	Genera snapshots periódicos del estado de la base de datos.	Registra cada operación de escritura realizada.
Pérdida de datos	Puede perder los cambios realizados desde el último snapshot.	Mínima pérdida de datos (hasta 1 segundo con everysec).
Rendimiento	Mayor rendimiento, menor impacto en las operaciones.	Menor rendimiento debido a las escrituras frecuentes en disco.
Tamaño de archivo	Más compacto y pequeño.	Generalmente más grande para el mismo conjunto de datos.
Tiempo de recuperación	Recuperación y reinicio más rápidos.	Recuperación más lenta al tener que reproducir operaciones.
Copias de seguridad	Ideal para backups y recuperación ante desastres.	Menos práctico para archivado de respaldos.
Durabilidad	Menor durabilidad.	Mayor durabilidad.
Legibilidad	Formato binario, difícil de inspeccionar.	Formato secuencial legible y fácil de analizar.
Reescritura	Genera snapshots completos.	Puede compactarse mediante reescritura automática del archivo.
Caso de uso recomendado	Cuando se prioriza rendimiento y backups periódicos.	Cuando se prioriza minimizar la pérdida de datos.

[Bibliografía sobre persistencia](#)

Persistencia en Redis

Recomendación

Recomendación práctica

Escenario	Opción recomendada
Caché pura	Sin persistencia o RDB
Alto rendimiento y tolerancia a perder algunos minutos de datos	RDB
Alta durabilidad y mínima pérdida de datos	AOF (everysec)
Sistemas productivos con datos importantes	RDB + AOF

Seguridad en Redis

Los principales problemas de seguridad en Redis son:

- Control de acceso.
- Seguridad del código.
- Entradas maliciosas.
- Diseñado para ser accedido por clientes de confianza dentro de entornos de confianza.
- No es una buena idea exponer la instancia directamente a Internet o acceder directamente al puerto o al socket.
- No está optimizado para la máxima seguridad sino para un máximo rendimiento y simplicidad.

Seguridad en Redis

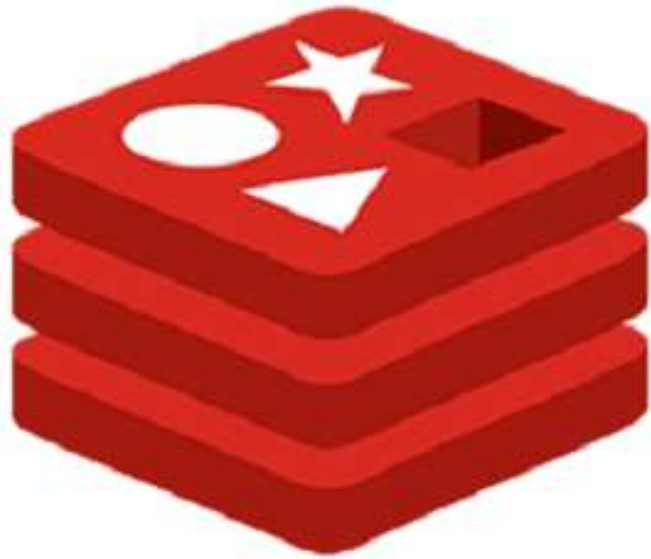
- En una aplicación web implementada utilizando Redis (base de datos, caché o sistema de mensajería) los clientes del front-end de la aplicación realizarán operaciones, pero en este caso la aplicación web media el acceso entre Redis y los clientes que no son de confianza.
- El acceso no confiable a Redis siempre debe estar mediado por una capa que implementa ACL, valide la entrada del usuario y decida qué operaciones realizar contra la instancia de Redis.

Seguridad en Redis

- Un cliente puede autenticarse enviando el comando AUTH seguido de la contraseña, cuando la capa de autorización está habilitada, Redis rechazará cualquier consulta realizada por clientes no autenticados.
- La capa de autenticación proporciona opcionalmente una capa de redundancia al mecanismo implementado.
- No soporta encriptado, por lo que se debe implementar una capa adicional de protección, como un proxy SSL.
- Como no posee el concepto de secuencias de escape en cadenas la inyección de código es imposible al igual que en los scripts Lua ejecutados por los comandos EVAL y EVALSHA.

[Bibliografía sobre seguridad](#)

Principales Comandos Redis



redis



Comandos (String)

Set: Elementos individuales

- SET: set key value
- GET: get key
- DEL: del key

Hash: permite poner varios pares de **key:value** para una key determinada.

- HSET: hset key subKey value
- HGETALL hgetall key
- HGET hget key subkey
- HKEYS hkeys key

Let's do it